Reusable Prompt Template Library for IntelliView Orchestrator

1. Introduction

Artificial Intelligence (AI) has become a fundamental component of modern recruitment systems by automating repetitive tasks and assisting recruiters in making informed hiring decisions. In AI-powered interview platforms such as IntelliView Orchestrator, Large Language Models (LLMs) are responsible for performing several critical workflows, including resume analysis, interview question generation, answer evaluation, candidate feedback generation, interview report summarization, and skill assessment.

As the number of AI-assisted workflows increases, creating prompts individually for each request can lead to inconsistencies, duplication, and maintenance challenges. Different prompt structures may produce varying outputs for similar tasks, making the system difficult to standardize and scale.

To address this challenge, reusable prompt templates provide a standardized approach for interacting with AI models. Instead of writing new prompts for every request, templates define a consistent structure with dynamic placeholders that can be populated with workflow-specific information. This improves response quality, promotes consistency across different interview sessions, and simplifies prompt maintenance as the system evolves.

This document presents a reusable prompt template library specifically designed for the IntelliView Orchestrator. The templates are intended to support common AI-driven interview workflows while remaining flexible enough to accommodate different job roles, experience levels, interview types, and evaluation criteria.

2. Objective

The primary objective of this project is to design a standardized and reusable prompt template library for the IntelliView Orchestrator that improves consistency, maintainability, and response quality across AI-assisted interview workflows.

The project aims to:

- Identify common AI-driven workflows within the interview system.
- Design reusable prompt templates for each workflow.
- Standardize prompt structures using configurable placeholders.
- Improve prompt maintainability through modular template design.
- Test each template using representative sample inputs.

- Document template usage, customization options, and best practices.
- Provide a prompt library that can be integrated into future versions of the IntelliView interview platform.

3. Project Scope

This prompt template library focuses on the primary AI workflows identified for the IntelliView Orchestrator. The templates are designed to support interview automation while remaining independent of any specific Large Language Model (LLM). Consequently, they can be adapted for use with Gemini, Grok, GPT-4o, or other compatible AI models with minimal modification.

The scope of this project includes the following workflows:

- Resume Analysis
- Interview Question Generation
- Answer Evaluation
- Candidate Feedback Generation
- Interview Report Summarization
- Skill Assessment

Each workflow is represented by a reusable prompt template that follows a common design structure, allowing the templates to be maintained, extended, and reused across multiple interview sessions.

4. Workflow Identification

The following AI-assisted workflows were identified as core components of the IntelliView interview system.

Workflow	Purpose
Resume Analysis	Extract candidate information, identify technical and soft skills, summarize experience, and determine strengths and missing competencies.
Interview Question Generation	Generate interview questions based on the candidate profile, required skills, job role, interview type, and difficulty level.
Answer Evaluation	Assess candidate responses using predefined evaluation criteria and provide structured scoring and feedback.

Candidate Feedback Generation	Generate constructive feedback highlighting strengths, weaknesses, and improvement recommendations after the interview.
Interview Report Summarization	Produce a comprehensive interview report summarizing candidate performance and overall hiring recommendation.
Skill Assessment	Evaluate demonstrated competencies and assign proficiency levels for required technical and professional skills.

5. Prompt Template Design Principles

To ensure consistency and maintainability, every prompt template in this library follows a standardized structure.

Each template contains the following components:

Component	Description
Template ID	Unique identifier for the prompt template.
Template Name	Name of the workflow supported by the template.
Objective	Purpose of the prompt template.
Workflow	IntelliView workflow where the template is used.
Required Inputs	Dynamic placeholders required by the template.
Prompt Template	Reusable prompt containing configurable placeholders.
Expected Output	Desired response structure generated by the AI model.
Sample Input	Example values supplied to the template.
Sample Output	Representative AI response generated from the sample input.
Usage Notes	Instructions for effectively using the template.
Best Practices	Recommendations for obtaining accurate and consistent results.

6. Standard Placeholder Definitions

The reusable templates utilize standardized placeholders that allow the same prompt to be applied across different interview scenarios.

Placeholder	Description
{candidate_name}	Candidate's full name.

{job_role}	Position for which the candidate is being interviewed.
{experience_level}	Fresher, Junior, Mid-Level, or Senior.
{resume_text}	Complete resume content of the candidate.
{job_description}	Job description or role requirements.
{skills}	Required or extracted technical and professional skills.
{interview_type}	Technical, HR, Behavioral, or Mixed interview.
{difficulty}	Difficulty level (Easy, Medium, Hard).
{question}	Interview question presented to the candidate.
{candidate_answer}	Candidate's response to the interview question.
{evaluation_criteria}	Criteria used for answer evaluation.

7. Reusable Prompt Template Library

PT-001: Resume Analysis

Template ID: PT-001

Template Name: Resume Analysis Template

Objective: To analyze a candidate's resume and extract structured information that can be used throughout the interview process, including interview question generation, skill assessment, and candidate evaluation.

Workflow: Resume Analysis

Purpose: This template enables the AI model to process a candidate's resume and generate a structured summary of the candidate's qualifications, technical competencies, work experience, educational background, strengths, and potential skill gaps relevant to the applied job role.

Required Inputs

Input Variable	Description
{candidate_name}	Candidate's full name
{job_role}	Position applied for
{job_description}	Job description or role requirements
{resume_text}	Complete resume content

Reusable Prompt Template

Role: You are an experienced technical recruiter and resume analyst responsible for evaluating candidate resumes for recruitment purposes.

Task: Analyze the provided resume and compare it with the given job role and job description.

Candidate Name: {candidate_name}

Job Role: {job_role}

Job Description: {job_description}

Resume: {resume_text}

Instructions:

1. Summarize the candidate profile.

2. Identify technical skills.

3. Identify soft skills.

4. Summarize education.

5. Summarize work experience.

6. Identify relevant projects.

7. Highlight strengths.

8. Identify missing skills compared to the job description.

9. Calculate an estimated job-role compatibility percentage.

10. Recommend whether the candidate should proceed to the interview stage.

Output Format:

Candidate Summary

Technical Skills

Soft Skills

Education

Experience

Projects

Strengths

Skill Gaps

Job Compatibility (%)

Recommendation

Expected Output

The AI should generate:

- Candidate Summary
- Technical Skills
- Soft Skills
- Education Summary
- Work Experience
- Project Summary
- Strengths
- Missing Skills
- Compatibility Percentage
- Interview Recommendation

Sample Input

Variable	Value
Candidate Name	Rahul Sharma
Job Role	Python Backend Developer
Job Description	Python, FastAPI, SQL, REST APIs, Docker
Resume	B.Tech CSE graduate with internships in Python development, FastAPI projects, SQL database management, and Docker deployment.

Sample Output

Candidate Summary: Rahul Sharma is a Computer Science graduate with internship experience in backend software development.

Technical Skills: Python, FastAPI, SQL, Docker, Git

Soft Skills: Communication, Problem Solving, Teamwork

Education: Bachelor of Technology in Computer Science

Experience: Python Backend Development Internship

Projects: Inventory Management API, Task Management System

Strengths: Strong Python fundamentals, Experience with REST APIs, Hands-on backend development

Skill Gaps: Limited cloud deployment experience, No CI/CD exposure

Job Compatibility: 88%

Recommendation: Proceed to Technical Interview

Usage Notes

- Use before interview scheduling.
- Suitable for technical and HR interviews.
- Can be used to personalize interview questions.
- Can support automated resume screening.

Best Practices

- Provide the complete resume text.
- Include the complete job description.
- Avoid abbreviating technical skills.
- Maintain consistent formatting across resumes.

PT-002: Interview Question Generation

Template ID: PT-002

Template Name: Interview Question Generation Template

Objective: Generate structured interview questions tailored to the candidate's profile, job role, interview type, and desired difficulty level.

Workflow: Interview Question Generation

Purpose: This prompt generates relevant interview questions by considering the candidate's resume, required skills, interview type, and job requirements. It also produces expected answers and follow-up questions to assist interviewers.

Required Inputs

Input Variable	Description
{job_role}	Position being interviewed
{skills}	Required technical skills
{experience_level}	Fresher, Junior, Mid-Level, Senior
{interview_type}	Technical / HR / Behavioral

{difficulty}	Easy / Medium / Hard

Reusable Prompt Template

Role: You are an experienced technical interviewer.

Task: Generate interview questions appropriate for the candidate profile.

Job Role: {job_role}

Skills: {skills}

Experience Level: {experience_level}

Interview Type:{interview_type}

Difficulty: {difficulty}

Instructions:

Generate five interview questions.

Arrange questions from easier to harder.

For each question provide:

Purpose

Expected Answer

Difficulty

Follow-up Question

Avoid repeated questions.

Ensure questions match the required skills.

Expected Output

For every question generate:

- Question
- Purpose
- Expected Answer
- Difficulty
- Follow-up Question

Sample Input

Variable	**Value**

Job Role	Python Backend Developer
Skills	Python, FastAPI, SQL, Docker
Experience Level	Fresher
Interview Type	Technical
Difficulty	Medium

Sample Output

Question 1: Explain the difference between a list and a tuple in Python.

Purpose: Evaluate Python fundamentals.

Expected Answer: Lists are mutable whereas tuples are immutable.

Difficulty: Easy

Follow-up: When would you choose a tuple instead of a list?

Usage Notes

- Can be used before every interview round.
- Supports multiple interview types.
- Suitable for adaptive interview systems.

Best Practices

- Provide accurate candidate skill information.
- Match the difficulty level to candidate experience.
- Avoid overly broad skill lists.
- Update required skills based on the job description.

PT-003: Answer Evaluation

Template ID: PT-003

Template Name: Answer Evaluaion Template

Objective: To evaluate a candidate's interview response based on predefined evaluation criteria and generate a structured assessment including score, strengths, weaknesses, and improvement suggestions.

Workflow: Answer Evaluation

Purpose: This template enables the AI model to objectively evaluate candidate responses by assessing technical correctness, conceptual understanding, communication clarity, problem-solving ability, and overall relevance to the interview question.

Required Inputs

Input Variable	Description
{question}	Interview question asked to the candidate
{candidate_answer}	Candidate's response
{evaluation_criteria}	Criteria used for evaluation
{job_role}	Position applied for
{difficulty}	Difficulty level of the question

Reusable Prompt Template

Role: You are an experienced technical interviewer responsible for evaluating interview responses objectively.

Task: Evaluate the candidate's answer based on the provided interview question and evaluation criteria.

Question: {question}

Candidate Answer: {candidate_answer}

Evaluation Criteria: {evaluation_criteria}

Job Role: {job_role}

Difficulty: {difficulty}

Instructions:

1. Evaluate the answer objectively.

2. Do not assume knowledge not present in the answer.

3. Reward partially correct answers where appropriate.

4. Provide constructive feedback.

5. Assign a score out of 10.

6. Return ONLY valid JSON.

Expected Output

```
{
  "score": 8,
```

"technical_accuracy": "High",

"communication": "Good",

"problem_solving": "Moderate",

"strengths": [],

"weaknesses": [],

"improvement_suggestions": [],

"overall_feedback": "",

"recommendation": ""

}

Sample Input

Variable	Value
Question	Explain the difference between multithreading and multiprocessing.
Candidate Answer	Multithreading allows multiple threads inside one process while multiprocessing creates separate processes.
Evaluation Criteria	Accuracy, Clarity, Examples, Communication
Job Role	Python Backend Developer
Difficulty	Medium

Sample Output

{

"score": 9,

"technical_accuracy": "High",

"communication": "Excellent",

"problem_solving": "Good",

"strengths": [

"Correct conceptual explanation",

"Clear communication"

],

"weaknesses": [

"Could include practical examples"

],
"improvement_suggestions": [
"Discuss Python GIL",
"Mention use cases"
],
"overall_feedback": "Strong understanding of the concept with minor scope for improvement.",
"recommendation": "Proceed to next question"
}

Usage Notes

- Used immediately after every candidate response.
- Suitable for technical, HR, and behavioral interviews.
- Supports automated scoring pipelines.

Best Practices

- Keep evaluation criteria consistent across interviews.
- Evaluate only the provided answer.
- Encourage constructive rather than punitive feedback.
- Store JSON responses for downstream analytics.

PT-004: Candidate Feedback Generation

Template ID: PT-004

Template Name: Candidate Feedback Generation Template

Objective: Generate personalized, constructive, and actionable feedback based on the candidate's overall interview performance.

Workflow: Candidate Feedback Generation

Purpose: This template summarizes interview performance by identifying strengths, weaknesses, learning opportunities, and actionable recommendations that can help the candidate improve.

Required Inputs:

Input Variable	Description

{candidate_name}	Candidate's full name
{evaluation_results}	Consolidated evaluation results
{job_role}	Position applied for

Reusable Prompt Template

Role: You are an experienced interview coach providing constructive candidate feedback.

Task: Generate professional post-interview feedback.

Candidate: {candidate_name}

Job Role: {job_role}

Evaluation Summary: {evaluation_results}

Instructions:

1. Be constructive and encouraging.

2. Highlight strengths.

3. Explain improvement areas.

4. Suggest learning resources or focus areas.

5. Return ONLY valid JSON.

Expected Output

```
{
  "overall_performance": "",
  "strengths": [],
  "areas_for_improvement": [],
  "recommended_learning": [],
  "final_feedback": ""
}
```

Sample Output

```
{
  "overall_performance": "Good",
  "strengths": [
    "Strong Python fundamentals",
    "Good communication",
```

"Logical problem solving"

],

"areas_for_improvement": [

"Database optimization",

"System design concepts"

],

"recommended_learning": [

"Practice SQL optimization",

"Study REST API architecture",

"Learn Docker deployment"

],

"final_feedback": "You demonstrated a solid technical foundation. Improving database optimization and system design knowledge will further strengthen your interview performance."

}

Usage Notes

- Used after the interview concludes.
- Can be shared directly with candidates.
- Supports personalized learning recommendations.

Best Practices

- Maintain a positive and professional tone.
- Base feedback only on interview evidence.
- Provide specific, actionable suggestions.

PT-005: Interview Report Summarization

Template ID: PT-005

Template Name: Interview Report Summarization Template

Objective: To generate a comprehensive interview report that summarizes the candidate's overall interview performance, technical competency, communication skills, behavioral observations, and hiring recommendation.

Workflow: Interview Report Summarization

Purpose: This template consolidates information collected throughout the interview process into a structured report that can be reviewed by recruiters, hiring managers, and interview panels.

Required Inputs

Input Variable	Description
{candidate_name}	Candidate's full name
{job_role}	Position applied for
{question_summary}	List of questions asked during the interview
{evaluation_results}	Consolidated evaluation results
{overall_score}	Final interview score

Reusable Prompt Template

Role: You are an experienced hiring manager responsible for preparing final interview reports.

Task: Generate a professional interview summary report.

Candidate Name: {candidate_name}

Job Role: {job_role}

Questions Asked: {question_summary}

Evaluation Summary: {evaluation_results}

Overall Score: {overall_score}

Instructions:

1. Summarize the interview objectively.

2. Highlight technical strengths.

3. Highlight communication strengths.

4. Mention improvement areas.

5. Provide an overall recommendation.

6. Return ONLY valid JSON.

Expected Output

```
{
  "candidate_name": "",
```

```
  "job_role": "",
  "overall_score": 0,
  "technical_performance": "",
  "communication": "",
  "behavioral_assessment": "",
  "strengths": [],
  "areas_for_improvement": [],
  "overall_summary": "",
  "hiring_recommendation": ""
}
```


Sample Input

Variable	Value
Candidate Name	Rahul Sharma
Job Role	Python Backend Developer
Question Summary	Python, SQL, FastAPI, Docker
Evaluation Results	Average score: 8.6/10
Overall Score	86

Sample Output


```
{
  "candidate_name": "Rahul Sharma",
  "job_role": "Python Backend Developer",
  "overall_score": 86,
  "technical_performance": "Strong",
  "communication": "Very Good",
  "behavioral_assessment": "Positive and collaborative",
  "strengths": [
    "Python programming",
    "REST API development",
    "Logical reasoning"
```

],
"areas_for_improvement": [
"Cloud deployment",
"CI/CD pipelines"
],
"overall_summary": "The candidate demonstrated strong backend development knowledge with good communication skills. Minor improvements in deployment technologies would further strengthen the profile.",
"hiring_recommendation": "Recommended for the next recruitment stage."
}

Usage Notes

- Used after completion of the interview.
- Supports recruiter decision-making.
- Can be stored in the candidate database.
- Useful for audit and future interview comparisons.

Best Practices

- Include evaluation data from all interview rounds.
- Avoid subjective or unsupported statements.
- Ensure recommendations are evidence-based.

PT-006: Skill Assessment

Template ID: PT-006

Template Name: Skill Assessment Template

Objective: To evaluate the candidate's proficiency across required technical and professional skills based on interview responses and assign competency levels.

Workflow: Skill Assessment

Purpose: This template identifies the candidate's demonstrated competency levels across predefined skills and highlights strengths, weaknesses, and recommended learning paths.

Required Inputs

Input Variable	Description

{candidate_name}	Candidate's full name
{job_role}	Position applied for
{required_skills}	Skills required for the role
{candidate_answers}	Candidate responses during the interview
{evaluation_results}	Interview evaluation summary

Reusable Prompt Template

Role: You are an experienced technical assessor.

Task: Assess the candidate's proficiency across the required skills based on the interview responses.

Candidate Name: {candidate_name}

Job Role: {job_role}

Required Skills: {required_skills}

Candidate Responses: {candidate_answers}

Evaluation Summary: {evaluation_results}

Instructions:

1. Evaluate each required skill independently.

2. Assign a proficiency level.

3. Justify every assessment.

4. Recommend learning areas where necessary.

5. Return ONLY valid JSON.

Expected Output

```
{
  "candidate_name": "",
  "job_role": "",
  "skill_assessment": [
    {
      "skill": "",
      "proficiency": "",
```

```
    "score": 0,
    "remarks": ""
   }
  ],
  "overall_skill_rating": "",
  "recommended_learning": []
}
```


Sample Input

Variable	Value
Candidate Name	Rahul Sharma
Job Role	Python Backend Developer
Required Skills	Python, FastAPI, SQL, Docker
Candidate Answers	Interview responses
Evaluation Results	Overall Score: 86

Sample Output


```
{
  "candidate_name": "Rahul Sharma",
  "job_role": "Python Backend Developer",
  "skill_assessment": [
   {
    "skill": "Python",
    "proficiency": "Advanced",
    "score": 9,
    "remarks": "Excellent understanding of Python fundamentals."
   },
   {
    "skill": "FastAPI",
    "proficiency": "Intermediate",
    "score": 8,
```

```
      "remarks": "Good understanding of API development."
    },
    {
      "skill": "SQL",
      "proficiency": "Intermediate",
      "score": 7,
      "remarks": "Needs more optimization knowledge."
    },
    {
      "skill": "Docker",
      "proficiency": "Beginner",
      "score": 5,
      "remarks": "Basic knowledge demonstrated."
    }
  ],
  "overall_skill_rating": "Intermediate",
  "recommended_learning": [
    "Docker",
    "CI/CD",
    "Cloud Deployment"
  ]
}
```


Usage Notes

- Used after answer evaluation.
- Supports hiring decisions.
- Can generate skill dashboards.
- Useful for candidate performance analytics.

Best Practices

- Assess only demonstrated skills.

- Avoid assumptions based on resume alone.
- Keep proficiency levels consistent.
- Update required skills according to the job description.

8. Prompt Library Summary

Template ID	Template Name	Workflow	Output Format
PT-001	Resume Analysis	Resume Analysis	Structured Sections
PT-002	Interview Question Generation	Question Generation	Structured Sections
PT-003	Answer Evaluation	Answer Evaluation	JSON
PT-004	Candidate Feedback Generation	Feedback Generation	JSON
PT-005	Interview Report Summarization	Report Generation	JSON
PT-006	Skill Assessment	Skill Assessment	JSON

9. Template Testing

9.1 Testing Objective

The purpose of template testing is to verify that each reusable prompt template produces consistent, relevant, and structured outputs when provided with representative inputs. Testing ensures that the templates can be reliably reused across different interview scenarios while maintaining response quality and minimizing ambiguity.

The templates were evaluated using representative candidate profiles and interview scenarios covering different workflows within the IntelliView Orchestrator. The generated responses were assessed for correctness, completeness, clarity, structure, and adherence to the expected output format.

9.2 Testing Methodology

The following testing methodology was adopted for evaluating each prompt template:

1. Define a representative sample input corresponding to the workflow.
2. Populate the reusable prompt template using the sample input values.

3. Execute the prompt using a Large Language Model (LLM).
4. Compare the generated response with the expected output format.
5. Evaluate the response using predefined quality metrics.
6. Record observations and identify possible improvements.

This methodology ensures consistent evaluation across all prompt templates.

9.3 Evaluation Criteria

Each template was evaluated using the following quality metrics.

Evaluation Metric	Description
Accuracy	Correctness of the generated response
Relevance	Relevance of the output to the given workflow
Completeness	Coverage of all requested information
Clarity	Ease of understanding and readability
Consistency	Uniformity of responses across multiple executions
Output Structure	Adherence to the required response format

9.4 Test Results

Template ID	Workflow	Accuracy	Relevance	Clarity	Structure	Overall Result
PT-001	Resume Analysis	Excellent	Excellent	Excellent	Excellent	Passed
PT-002	Interview Question Generation	Excellent	Excellent	Excellent	Excellent	Passed
PT-003	Answer Evaluation	Excellent	Excellent	Very Good	Excellent	Passed
PT-004	Candidate Feedback Generation	Very Good	Excellent	Excellent	Excellent	Passed
PT-005	Interview Report Summarization	Excellent	Excellent	Excellent	Excellent	Passed
PT-006	Skill Assessment	Excellent	Excellent	Very Good	Excellent	Passed

9.5 Testing Observations

The testing process demonstrated that the reusable prompt templates consistently generated structured and relevant responses for the selected interview workflows. Templates designed for resume analysis and interview question generation effectively produced detailed outputs that aligned with the provided candidate information and job requirements.

The answer evaluation, candidate feedback, interview report summarization, and skill assessment templates successfully generated structured responses in the expected JSON format. This structured output improves compatibility with backend systems by simplifying data parsing and reducing the need for additional post-processing.

Overall, the prompt templates satisfied the primary objectives of consistency, maintainability, and response quality while remaining adaptable to different interview scenarios.

10. Usage Guidelines

The prompt templates are intended to be reused across different interview workflows within the IntelliView Orchestrator. Each template should be customized by replacing the predefined placeholders with workflow-specific values before being submitted to the selected AI model.

The following general procedure should be followed when using the templates:

1. Select the appropriate prompt template based on the required interview workflow.
2. Replace all placeholder variables with actual candidate or interview information.
3. Verify that the provided inputs are complete and accurate.
4. Submit the completed prompt to the configured AI model.
5. Review the generated response for completeness and correctness.
6. Store or process the response according to the corresponding workflow requirements.

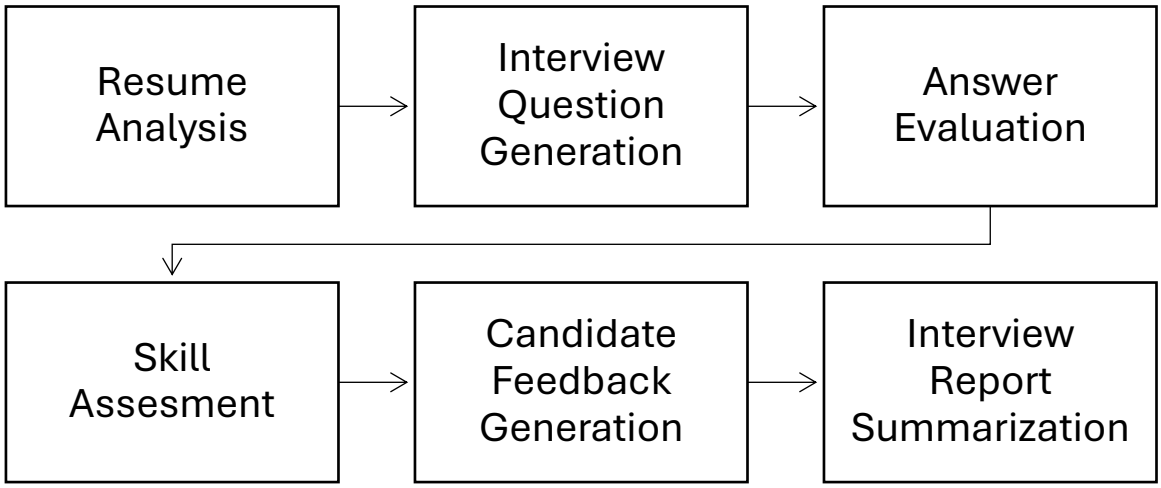
10.1 Placeholder Customization

Each prompt template uses standardized placeholders that can be dynamically replaced during execution.

Placeholder	Example Value
{candidate_name}	Rahul Sharma
{job_role}	Python Backend Developer
{experience_level}	Fresher
{resume_text}	Candidate Resume
{job_description}	Backend Developer Job Description

{skills}	Python, FastAPI, SQL
{question}	Explain REST APIs.
{candidate_answer}	Candidate Response
{evaluation_criteria}	Accuracy, Clarity, Problem Solving

10.2 Recommended Workflow



11. Best Practices

The following best practices are recommended when using reusable prompt templates within AI-powered interview systems:

- Clearly define the AI's role before specifying the task.
- Provide sufficient contextual information to improve response quality.
- Use standardized placeholders instead of hardcoded values.
- Ensure that all required input variables are populated before execution.
- Request structured outputs whenever possible to facilitate system integration.
- Avoid ambiguous instructions and maintain concise prompt wording.
- Keep prompt templates modular so they can be reused across multiple workflows.
- Validate AI-generated responses before using them for hiring decisions.
- Periodically review and update prompt templates to reflect changing recruitment requirements and AI model capabilities.
- Test templates with diverse candidate profiles to ensure consistent performance across different interview scenarios.

12. Future Improvements

The current prompt template library establishes a reusable framework for common interview workflows. Future enhancements can further improve flexibility, scalability, and integration within the IntelliView Orchestrator.

Potential improvements include:

- Support for multilingual interview workflows.
- Adaptive prompt generation based on candidate performance during the interview.
- Integration with Retrieval-Augmented Generation (RAG) for organization-specific interview questions.
- Automated prompt optimization using evaluation feedback.
- Version-controlled prompt repositories for collaborative development.
- Support for additional interview formats such as coding assessments and case-study evaluations.
- Enhanced personalization using candidate history and previous interview performance.
- Integration with analytics dashboards to monitor prompt effectiveness and response quality over time.

13. Conclusion

This project presented the design and documentation of a reusable prompt template library for the IntelliView Orchestrator. The library addresses six core interview workflows, namely Resume Analysis, Interview Question Generation, Answer Evaluation, Candidate Feedback Generation, Interview Report Summarization, and Skill Assessment.

Each prompt template was designed using a standardized structure with configurable placeholders to improve consistency, maintainability, and response quality. Representative testing demonstrated that the templates generated structured and relevant outputs suitable for AI-assisted interview workflows.

The resulting prompt library provides a modular foundation that can be extended as the IntelliView platform evolves. By promoting standardized prompt engineering practices, the proposed templates simplify prompt maintenance, enhance interoperability across AI models, and contribute to a more reliable and scalable interview orchestration system.